查找 7

章节脉络



查找结构要回答两个问题：怎样缩小候选范围，插入或删除后怎样保持这种缩小范围的能力。顺序查找只依赖逐项比较；折半查找依赖顺序存储且有序；搜索树把有序关系放进分支；散列表则用关键字计算位置。外存索引还要考虑一次读入能取得多少个关键字。

错项排除·先决定 ASL 的事件空间

成功概率分配给已有记录，失败概率可能分配给关键字间隙或初始散列地址。先写清每项概率再计平均；不能看到有效记录有 n 个，就把所有失败长度都除以 n。

7.1 本章知识结构

- **查找对象与代价**：主关键字与次关键字；静态动态查找表；成功失败事件；概率与平均查找长度；比较次数及外存页数。
- **线性表上的查找**：无序及有序顺序查找；哨兵与失败边界；分块索引与块长；折半查找的闭区间；判定树与失败外部位置。
- **二叉搜索与平衡**：BST 全部子树区间；查找插入删除；AVL 平衡因子；LLRRLRRL 旋转；红黑性质及黑高；红黑插入删除修复。
- **多路外存索引**：m 阶 B 树容量；分裂借位合并；根升降与叶等深；B 树高度界；B+ 树索引及叶链；点查范围与读盘次数。
- **散列查找**：散列函数构造；冲突及同义词；链地址法；开放定址探查序列；删除标记及重复判断；装填因子与 ASL。
- **结构选择的条件**：有序与随机定位；动态更新及维护；平衡性与最坏保证；外存页分支数；等值与范围检索。

按“候选范围怎样缩小、失败怎样判定、更新怎样维护结构”三条线阅读。ASL 计算始终先写清事件集合、概率和计数口径，再执行实际查找路径。

7.2 关键字、成功失败与平均查找长度

定义 7.2.1 (查找表、关键字与动态性)

查找表是同一类型记录的集合，**关键字**是用于识别记录的数据项。主关键字唯一标识记录，次关键字可能对应多条记录。只进行查找、不改变记录集合的表称为静态查找表；还需要插入、删除记录的表为动态查找表，不能仅凭 “采用数组” 判断是静态还是动态。

查找长度通常指一次查找中关键字的比较次数，平均查找长度为 $\mathrm{ASL}=\sum_i p_i C_i$，其中 p_i 为某一查找事件的概率，C_i 为该事件的比较次数。成功 ASL 在表内各条记录之间分配概率；失败 ASL 在不存在的关键字可能落入的失败区间或初始散列地址之间分配概率，分母未必是表长。

树形查找中，访问一个含多关键字的 B 树结点可能只触发一次外存读，却在内存中比较多个关键字。因此 “读盘次数”“访问结点数”“关键字比较次数” 不是同一个量，题目要求哪个就统计哪个。

条件辨析·同结构名也要看有效范围

哨兵顺序查找的 SSTable 用 1 至 n，折半查找课件用 0 至 n-1。elem 和 TableLen 含义相同，有效下标范围由具体算法规定。

7.3 顺序查找、分块查找与提前结束

无序表从一端逐项比较，成功时停止，扫描完仍未找到则失败。等概率成功时，平均比较 $(n+1)/2$ 次；失败通常比较 n 次。哨兵可以统一循环终止条件、减少边界判断，但不会改变比较次数的数量级。若计入哨兵自身的关键字比较，次数还须按代码约定加上相应一项。

递增有序表从头查找时，当前值等于目标时应报告成功，当前值**大于**目标才可提前失败；把 “不小于” 一概判为失败会把相等的目标漏掉。若仍逐个向后比较，等概率成功 ASL 与无序表相同，只是失败可能提前结束；有序并不自动意味着每种查找都使用折半。

7.3.1 顺序查找的哨兵实现

```c
typedef struct {
    ElemType *elem;
    int TableLen;
} SSTable;
int Search_Seq(SSTable ST, ElemType key) {
    ST.elem[0]=key;
    int i;
    for (i=ST.TableLen; ST.elem[i]!=key; --i) ;
    return i;
}
```

本段将有效记录放在 `elem[1..TableLen]`，另有可写的 0 号哨兵单元。从后向前查找，命中返回该下标，失败必停在 0 并返回 0。函数虽按值传入结构体，其指针成员仍指向同一数组，所以写哨兵会修改原数组的 0 号单元。空表也必须存在可写哨兵位置。

等概率成功时，比较次数为 $1,2,\ldots,n$，平均 $(n+1)/2$；失败比较 n 个实际记录再比较哨兵，共 $n+1$ 次。若题目只数与实际记录的比较，则最后一次哨兵另算。哨兵节省每轮边界检测，并没有省去线性扫描。

课件的非等概率例，六条记录依次命中概率为 0.15、0.05、0.10、

0.40、0.28、0.02，从前向后查的成功 ASL 为 3.67。按命中概率非增序排列后，为 0.40、0.28、0.15、0.10、0.05、0.02，ASL 降为 2.18。交换相邻两条逆概率记录可使代价降低，故这种排序适合无须维持关键字顺序的静态顺序查找表。

分块查找把表分成若干块，块内可无序，块间有序：前一块的最大关键字小于后一块所有关键字。索引保存每块最大关键字和块起始位置，先定位块，再在块内顺序找。索引查找若分别采用顺序查找与折半查找，效率并不相同。

索引保存块的最大值时，应定位**第一个最大关键字不小于目标**的块。若对索引折半后没有命中相等值，循环结束的 `low` 仍可指向目标所属块；只有 `low` 超过最后一个索引项，才能直接判断目标不在表内。例如块最大值为 `10,20,30`，查 17 应进入最大值 20 对应的块，不是因索引无 17 而失败。

块内有序并非分块查找的必要条件。动态表可以把每块组织成链表以方便更新，但索引最大值和块间范围仍须维护。

不变量·mid 比较后必须排除

闭区间中，相等已处理后，目标只能在 mid 左边或右边。若更新 high=mid 且区间仅两个元素，可能反复取得同一 mid 而不缩小；循环条件与更新式必须配套。

7.3.2 例 7-1　两级顺序查找怎样选择块长

例 7.3.1 (配套案例)

【配套例题】查找表有 400 个元素，查找每个元素的概率相同，采用均匀分块。先顺序查找确定所在块，块内也顺序查找。为使查找效率最高，每块包含元素个数应为（　）。

A. 8　B. 10　C. 20　D. 25

解析。设每块 s 个，共 $b = 400/s$ 块。查成功时，索引平均比较 $(b+1)/2$ 次，块内平均比较 $(s+1)/2$ 次，总计

$$\mathrm{ASL} = \frac{400/s + s}{2} + 1.$$

两项乘积固定，$400/s + s$ 在 $s = \sqrt{400} = 20$ 时最小，答案为 **C**。四个选项均能整除 400，无需处理尾块。若索引改为折半，这个 “平方根块长” 的推导不再对应实际代价。

7.4 折半查找与判定树

折半查找要求按关键字有序且能快速定位中点。本章教材代码在闭区间 `[low,high]` 中查找，取 `mid=(low+high)/2`，向下取整。它按 `elem[0..TableLen-1]`存放有效记录，与上一节哨兵顺序查找的有效范围不同。目标比中点小时令 `high=mid-1`，比中点大时令 `low=mid+1`；中点已经比较过，更新区间时不能把它又留在候选范围内。

```c
int Binary_Search(SSTable L, ElemType key) {
    int low=0, high=L.TableLen-1, mid;
    while (low<=high) {
        mid=(low+high)/2;
        if (L.elem[mid]==key) return mid;
        else if (L.elem[mid]>key) high=mid-1;
        else low=mid+1;
    }
    return -1;
}
```

课程连接·比较少不代表访存少

折半查找在数组上可常数时间定位中点；链表上虽能设计相同的比较决策，走到中点还要访问中间结点。算法分析要把抽象比较与实际存储访问分开。

成功查找的比较次数等于对应内部结点所在层次。失败发生在候选区间为空时，空位置本身通常不做关键字比较，因此沿途比较次数等于经过的内部结点数。取上中点或下中点会改变具体路径，数量级仍是 $O(\log n)$。

对长度 11 的递增表 `7,10,13,16,19,29,32,33,37,41,43`，第一次比较中间项 29，查 33 依次比较 29、37、32、33；查 12 依次比较 29、13、7、10，然后 `low>high` 失败。

判定树以每次比较的中点为内部结点。向下取中点时，偶数长度区间分出的右子树比左子树多 1 项，奇数长度时两侧一样多；因此判定树高度平衡，却未必是完全二叉树。上述 11 项表，成功结点分布为第 1 层 1 个、第 2 层 2 个、第 3 层 4 个、第 4 层 4 个，成功 ASL 为 3。12 个失败区间中 4 个需 3 次比较、8 个需 4 次，失败 ASL 为 $11/3$。失败外部结点所在层次比它实际比较的内部结点数多 1。

若把中点改为向上取整，左右规模关系相反，具体 ASL 和搜索轨迹须按新判定树计算。处理可能很大的整型下标时，`low+(high-low)/2` 可避免求和溢出；本章教学表长取可安全相加的范围。

7.4.1 例 7-2　失败查找的最大比较次数

例 7.4.1 (配套案例)

【配套例题】长度为 16 的顺序表 L 按关键字有序，采用折半查找，查找一个不存在的元素，关键字比较次数最多为（　）。

A. 4　B. 5　C. 6　D. 7

解析。用从 1 开始的位序、向下取中点，查找一个大于全部元素的值：区间依次为 `[1,16]→[9,16]→[13,16]→[15,16]→[16,16]→ 空`，比较位置为 8、12、14、15、16，共 5 次，答案为 **B**。$\log_2 16 = 4$ 不是这里的最大比较次数；还需要最后一次区分第 16 个元素与它右侧的失败区间。

同一约定下，长度 11 时查第 11 个元素，比较位序为 6、9、10、11；长度 10、关键字依次 `b,c,d,e,f,g,q,r,s,t` 时查 b，比较 f、c、b。不能在不同题中改用向上取中点。

有序链表即使能逐步寻找中点，也没有数组的常数时间随机定位能力；普通折半查找的对数比较次数并不自动带来对数总访问时间。无序数组不满足排除半区的条件；静态链表虽然用数组存结点，逻辑顺序也不保证按下标连续。

> **先写查找区间不变式再写边界**
>
> 闭区间折半查找循环开始时，若目标存在，它必在 `[low,`

> `high]`。比较过 `mid` 并排除相等后，下一区间必须去掉 `mid`；每轮区间严格缩小，`low>high` 才判失败。改为半开区间时，终止条件与更新式要一起改，不能从两份模板各抄一半。

7.5 二叉搜索树的区间不变式

本书二叉搜索树（BST）默认关键字互异，左子树所有值小于根，右子树所有值大于根，左右子树分别满足同样性质。只检查根与两个直接孩子还不够；一个较深后代也可能越过祖先设定的区间。中序遍历严格递增是等价的整体判断。

查找从根出发，每次比较后进入相应子树；插入沿相同路径，到空位置新建叶结点。比较次数最多为树高，树可能退化成链，因此普通 BST 最坏为 $O(n)$，不能无条件写 $O(\log n)$。

删除叶结点直接断开双亲链接；只有一个孩子时让孩子接替位置；有两个孩子时，可把中序前驱（左子树最大值）或后继（右子树最小值）移到当前位置，再删除那个原结点。被选的前驱不会有右孩子，后继不会有左孩子，最后归结到前两种情况。若题目要求保留结点地址而非只保留关键字，那么复制数据与整体移接就是两种不同的实现，指针的改法也随之不同。

验证 BST 可以中序扫描并比较相邻值，也可以给每个结点传递允许区间 `(lower,upper)`。左递归把上界收紧到当前值，右递归把下界提高到当前值；这样不会漏掉“孩子合格但孙子越过祖先”的错误。当边界值可能等于整数类型极值时，用更宽类型或显式“有无边界”标记，不要随意把一个合法值当作无穷。

定理 7.5.1 (BST 的中序判据)

关键字互异时，二叉树是递增 BST，当且仅当其中序遍历严格递增。

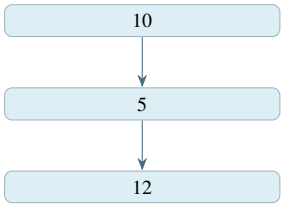
把全子树区间与中序顺序连接

若为 BST，左子树各值小于根、右子树各值大于根，分别递归中序，整体严格递增。反之，中序严格递增使每个结点左子树的全部访问项均小于它，右子树全部项均大于它；递归对子树也成立，故满足 BST 的全局定义。证明涉及全部后代，不能简化为只比直接孩子。

□

最小反例·孩子都合格，孙子越界

根 10，左孩子 5，5 的右孩子 12。每个直接孩子比较都合格，但 12 位于 10 左子树而大于 10，违反全局区间；检验 BST 需中序或上下界递归。



10 的左子树里出现 12：直接父子比较仍可能全对。

7.5.1 BST 查找、插入和删除的基本路径

```c
typedef struct BSTNode {
    int key;
    struct BSTNode *lchild, *rchild;
} BSTNode, *BSTree;
BSTNode *BST_Search(BSTree T, int key) {
    while (T!=NULL && key!=T->key) {
        if (key<T->key) T=T->lchild;
        else T=T->rchild;
    }
    return T;
}
int BST_Insert(BSTree &T, int k) {
    if (T==NULL) {
        T=(BSTree)malloc(sizeof(BSTNode));
        T->key=k;
        T->lchild=T->rchild=NULL;
        return 1;
    } else if (k==T->key) return 0;
    else if (k<T->key) return BST_Insert(T->lchild,k);
    else return BST_Insert(T->rchild,k);
}
void Create_BST(BSTree &T, int str[], int n) {
    T=NULL;
    for (int i=0; i<n; ++i) BST_Insert(T,str[i]);
}
```

删除的递归返回值表示 “删除后这棵子树的新根”，从而统一处理删除整树根与内部结点。

```
erase(t,key):
    if t为空: return 空
    if key<t.key: t.lchild=erase(t.lchild,key)
    else if key>t.key: t.rchild=erase(t.rchild,key)
    else:
        if t.lchild为空: 保存t.rchild，释放t，返回保存值
        if t.rchild为空: 保存t.lchild，释放t，返回保存值
        q=t.rchild；沿q.lchild找到右子树最小结点
        t.key=q.key（记录有附加字段时一并复制）
        t.rchild=erase(t.rchild,q.key)
    return t
```

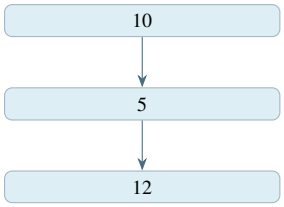
沿路每层常数工作，查找/插入/删除为 $O(h)$；递归删除额外栈 $O(h)$。这里复制记录后释放后继结点，不保留所有原结点地址；若题目约束结点身份，应改用移接方案。

`BST_Search` 返回实际结点指针或 `NULL`，迭代实现额外空间 $O(1)$；`BST_Insert` 返回 1 表示插入新关键字，返回 0 表示重复而未插入，引用 `T` 使空子树能够接回新结点。这里沿用教材默认内存申请成功的教学前提，实际程序还需处理分配失败。

课件按 `50,66,60,26,21,30,70,68` 构造 BST，根为 50，左子树 26 的孩子为 21、30，右子树 66 的孩子为 60、70，70 的左孩子为 68。按 `50,26,21,30,66,60,70,68` 插入也得到相同树，因为各子树内相对插入次序保持不变。把递增关键字依次插入则可能形成单右链，体现普通 BST 对插入次序的依赖。

最小反例·孩子都合格，孙子越界

根 10，左孩子 5，5 的右孩子 12。每个直接孩子比较都合格，但 12 位于 10 左子树而大于 10，违反全局区间；检验 BST 需中序或上下界递归。



10 的左子树里出现 12：直接父子比较仍可能全对。

7.6 AVL：先定位最小失衡子树

错项排除·删除传播与插入不同

插入修复最小失衡子树后一般恢复原高度，更高祖先不再受本次增高影响。删除后局部修复可能继续降低高度，仍须向上；一处调整并非所有 AVL 更新的共同规则。

AVL 是每个结点左右子树高度差绝对值不超过 1 的 BST。令平衡因子为“左高减右高”，合法值为 −1、0、1。插入后沿查找路径向上更新高度，找到离插入位置最近的失衡祖先 A，再根据新结点相对 A 及其较高孩子的位置分类。

LL（左孩子的左侧增高）对 A 右旋；RR 对 A 左旋；LR 先对 A 的左孩子左旋，再对 A 右旋；RL 先对 A 的右孩子右旋，再对 A 左旋。名称描述失衡路径，不能把 “LL” 误当作 “向左旋”。旋转必须保持中序次序：右旋时原左孩子 B 升为局部根，B 原来的右子树移成 A 的左子树，因为它的值介于 B 与 A 之间。

以下数值树用于观察旋转机制。插入前各树高度均为 3；灰底新结点插入后，最小失衡子树高度升为 4。单旋图同时保留需要转接的中间子树：LL 中 40 由 30 的右子树改接为 50 的左子树，RR 中 40 的转接方向对称。

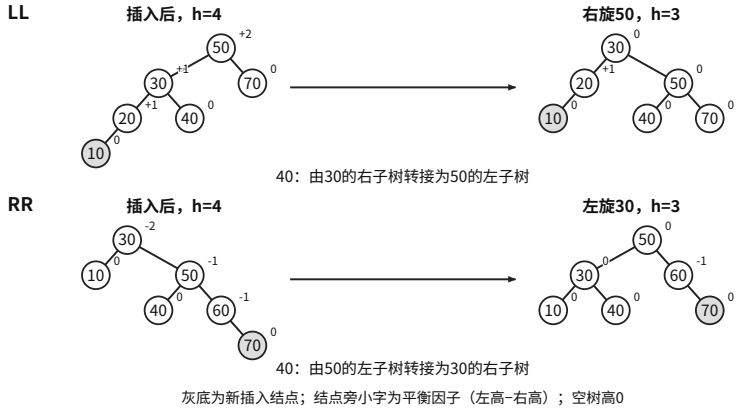


Figure 7.1: LL/RR 各一次旋转；结点旁为平衡因子，非空中间子树随旋转转接

LR 图在 70 的左孩子 30 的右侧插入 40，先左旋 30，再右旋 70；第一步把 40 转接为 30 的右孩子。RL 图在 50 的右孩子 90 的左侧插入 80，先右旋 90，再左旋 50；第一步把 80 转接为 90 的左孩子。**第一步后的树仍可能失衡**，双旋应检查完整两步，不能在中间态停下。四例都保持同一组关键字的升序中序，最终高度恢复为 3，所有平衡因子回到 −1、0、1。

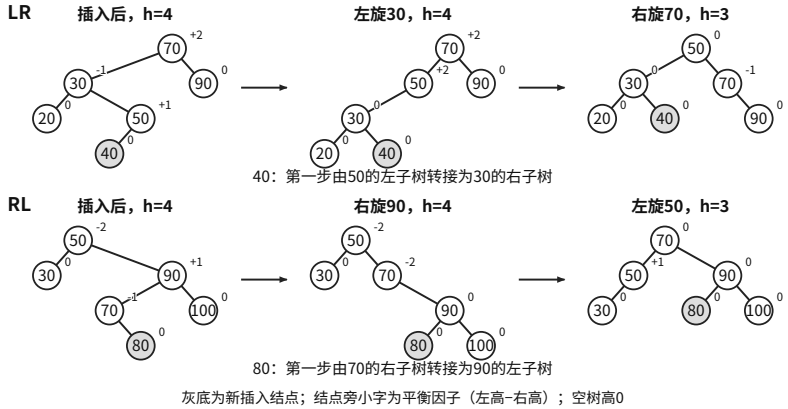


Figure 7.2: LR/RL 依次给出插入后、第一旋后和第二旋后三个状态

7.6.1 例 7-3　连续插入时在哪里旋转

例 7.6.1 (配套案例)

〖7.3-003〗把 34、23、15、98、115、28、107 依次插入初始为空的平衡二叉搜索树，要求每次插入后保持 AVL 性质，给出每步结果。

条件辨析·孩子平衡因子 0

这个分支在删除中常出现，可用一次单旋修复，且局部高度可能保持不变。插入口诀不能代替删除的高度传播判断。

解析。以下用 **根（左子树,右子树）** 表示局部结构，⌀ 表示空。

插入	失衡处与调整	调整后整树
34	无	34
23	无	34(23,∅)
15	34 处 LL，右旋	23(15,34)
98	无	23(15,34(∅,98))
115	34 处 RR，左旋	23(15,98(34,115))
28	23 处 RL，先右旋 98，再左旋 23	34(23(15,28),98(∅,115))
107	98 处 RL，先右旋 115，再左旋 98	34(23(15,28),107(98,115))

插入 28 时，不能只旋转它的双亲 34：34 仍平衡，最先失衡的是 23，新增路径是 “23 的右—98 的左”，故为 RL。每次调整后复查中序始终为已插入关键字的升序，避免只把树形画平衡而破坏搜索关系。

最小失衡子树调整后通常恢复到插入前的高度，更高的祖先不会再因这次插入而失衡。删除则可能使子树高度继续缩短，需要一路向上检查，不能把插入时 “一处修复即可” 的结论直接用于删除。AVL 最小结点数满足 $N(h)=1+N(h-1)+N(h-2)$，由此树高是 $O(\log n)$，查找与更新路径才有对数保证。

固定高度下的形态数则是另一个量。按本书的规定，空树高 0、单根高 1，令 $C(h)$ 为恰高 h 的 AVL 形态数，$C(0)=C(1)=1$；恰高 2 的左右高度只能是 `(1,1)`、`(1,0)`、`(0,1)`，所以 $C(2)=3$。恰高 3 时，左右高度为 `(2,2)`、`(2,1)`、`(1,2)`，故 $C(3)=3^2+2\times 3\times 1=15$，对应〖7.3-013〗。这不等于固定结点数的卡特兰计数，也不等于最小结点数递推。

7.6.2 AVL 删除怎样向上传递

删除先按 BST 规则处理实际被移除的结点，再沿其原祖先路径向上更新高度。失衡结点 A 若左高，考察左孩子 B 的平衡因子；右高情形完全对称。

A 的平衡因子	较高孩子的平衡因子	调整
$+2$	B 为 $+1$ 或 0	对 A 右单旋
$+2$	B 为 -1	先左旋 B，再右旋 A
-2	B 为 -1 或 0	对 A 左单旋
-2	B 为 $+1$	先右旋 B，再左旋 A

孩子平衡因子为 0 是删除中需要单独识别的情形：旋转后局部树高可能保持为删除前的高度，于是上层高度不再改变；其他情形旋转后仍可能比原先矮 1 层，需要继续向上。只看 “LL/RR” 口诀而漏掉孩子为 0，容易选错旋转或错误停止。

最小示例：`4(2(1,3),5)`删除 5 后，根 4 左高 2，左孩子 2 的平衡因子 0。右旋得到 `2(1,4(3,空))`，树高仍为 3，所有结点已平衡。若删除使下层子树真正矮 1 层并影响更高祖先，则应在更高处继续修复。整个查找、删除和沿祖先路径调整的时间为 $O(\log n)$。

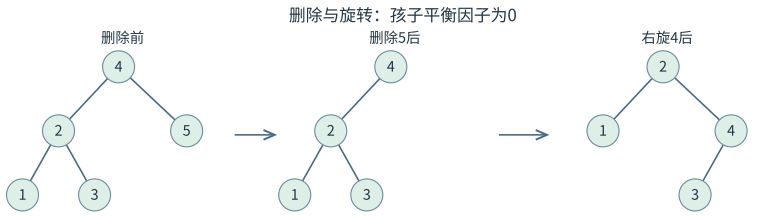


Figure 7.3: 删除 5 造成左高，孩子 2 平衡因子为 0；右旋后高度保持 3

不变量·旋转保留中序次序

右旋时 B 原右子树 T 的关键字介于 B 与 A 之间，转接为 A 左子树仍符合区间。若忘记保存 T，会丢失子树；若只交换数值，结点身份和其他链接语义也可能改变。

7.6.3 例 7-4　中序为降序时搜索方向怎样变

例 7.6.2 (配套案例)

【配套例题】一棵无重复关键字的 AVL 树，中序遍历得到降序序列。正确叙述是（　）。

A. 根的度一定为 2　　B. 最小元素一定是叶结点

C. 最后插入元素一定是叶结点　　D. 最大元素一定没有左子树

解析。本题的搜索方向与通常递增 BST 相反：左子树全大于根，右子树全小于根。最大值就是一直向左走到的那个结点，它没有左子树，答案为 **D**。AVL 约束不保证根一定有两个孩子；最小值结点可以有左孩子；新结点虽然最初作为叶插入，旋转后可能成为内部结点。不要用平衡性代替关键字的方向条件。

7.6.4 单旋代码为何必须保存中间子树

```
void R_Rotate(BSTree &p) {
    BSTNode *lc=p->lchild;
    p->lchild=lc->rchild;
    lc->rchild=p;
    p=lc;
}
void L_Rotate(BSTree &p) {
    BSTNode *rc=p->rchild;
    p->rchild=rc->lchild;
    rc->lchild=p;
    p=rc;
}
```

这里以引用 `p` 接回旋转后局部根，调用时将原双亲的相应孩子域传入；旋转整树时传入整树根指针。若结构还保存平衡因子或高度，须在旋转后依实际子树重新维护，先处理下降结点，再处理提升结点。右旋中 $B<T<A$ 的区间关系保证 T 接到 A 左侧仍合法；只交换两个关键字不等同于链接旋转。

AVL 插入可在 BST 递归返回时逐层更新高度，若平衡因子超界，则按新增关键字落在重侧孩子哪一边选择 LL/RR/LR/RL。最少结点递推的边界为 $N(0)=0, N(1)=1$；删除时子树变矮可能继续向上传播，应沿整条祖先路径复查。

7.7 红黑树：颜色约束怎样限制高度

错项排除·红叔要先变色

红双亲配红叔时，旋转不是第一选择：把双亲与叔变黑、祖父变红，将冲突上移，再保证根黑。黑叔的内外侧旋转情形不能覆盖这个状态。

红黑树仍是 BST，另为结点标红或黑，采用以下约定：根为黑；所有空孩子视为黑色 NIL 叶；红结点的孩子均为黑；从任一结点到其所有后代 NIL 叶的路径，黑结点数相同。NIL 不是普通数据结点，画图时可以省略，但计算黑高时必须把它计入。

相邻红结点不允许，所以一条路径中红结点数不会超过相应黑结点数。本书把黑高 b 定义为：从当前结点向下到任一 NIL 的路径上，不计当前结点，但把终点 NIL 计入，统计所含的黑结点数；NIL 自身的黑高记 0。黑高为 b 的子树至少含 2^b-1 个内部结点。于是最长路径的长度至多约为最短路径的两倍，整体高度为 $O(\log n)$。它不要求每个结点左右高度差至多 1，比 AVL 的局部高度约束宽。

按上述黑高 b 约定，根为黑的非空红黑树内部结点数满足 $2^b-1 \le n \le 2^{2b}-1$。下界可由 b 层全黑满树达到；上界由黑、红两色交替并填满共 $2b$ 个内部层达到。例如根黑高 2 时，内部结点数最少 3、最多 15。NIL 不计入内部结点数，不能把其数量也加到这两个界中。

插入先按 BST 加入，通常把新结点染红，以免立即改变各路径黑结点数。双亲黑则完成；双亲红时，看祖父另一侧的叔结点：

- 叔为红：双亲与叔变黑、祖父变红，把问题上移到祖父，最后保证根黑。
- 叔为黑（含 NIL）：若形成内侧折线，先在双亲处旋转为外侧；再在祖父处反向旋转，并交换提升结点与原祖父的颜色，消除红红相邻。

例如黑根 10、红左孩子 5，插入红 1，叔为空黑，形成 LL：右旋 10 后 5 成为黑根，1 与 10 为红，黑路径数保持一致。若原来还存在红右孩子 15，则插入 1 先使 5、15 变黑，祖父 10 暂红，最后根 10 恢复黑；这个情境应变色，不应直接套前一个旋转。

7.7.1 按课件序列观察红黑插入

从空树依次插入 `20,10,5,30,40`：20 作为根染黑；10 作为红左孩子接入；插入 5 使红 10 与红 5 相邻，叔为黑 NIL，属于 LL，右旋 20 并换色，得到黑根 10、红孩子 5 和 20。插入 30 时父 20 与叔 5 均红，把 5、20 染黑，祖父 10 暂红后恢复根黑；插入 40 时父 30 红、叔为空黑，发生 RR，对 20 左旋、把 30 染黑 20 染红。

最终为黑根 10，左孩子 5 黑，右孩子 30 黑且有红孩子 20 和 40。每步既保持中序递增，又保持所有通往 NIL 的黑高相等。课件后续的 `57,3,2,4,35,25,18,22,23,24,19,18` 用于继续观察向上传播、内侧双旋及重复值处理；若本章按互异关键字的 BST 接口实现，末尾重复 18 应返回未插入。题目允许重复时，先说明重复项安放或计数规则。

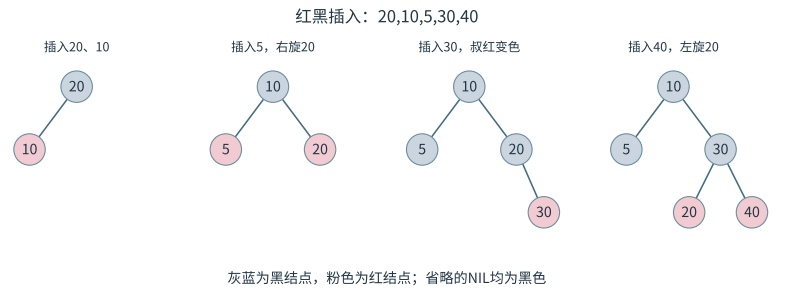


Figure 7.4: 从 20、10、5 到 30、40：黑叔旋转和红叔变色分别处理

修复的旋转次数和沿祖先链变色的次数应分别统计。红黑插入至多作常数次旋转，颜色调整可能一路传播到根；整个插入仍需先搜索且最坏 $O(\log n)$。红黑树不意味着一次插入的所有工作都为 $O(1)$。

删除时先按 BST 把待删结点化为至多只有一个非空孩子的情形。删红不减少路径黑高；删黑后，若有红孩子接替，可把接替孩子染黑。若替代位置是黑或 NIL，则沿该分支缺少一个黑结点，需要根据兄弟与其孩子颜色修复。

设缺黑位置 x 是双亲 p 的左孩子，兄弟 w 为 p 的右孩子。w 的左孩子为近侧、右孩子为远侧，均相对 x 而言；NIL 按黑处理。按表中的条件逐步转换，不能把 “红兄弟” 当作已经完成修复。

不变量·B 树合法性有五项

每次操作完成后同时检查结点内有序、孩子区间、键数上下限、孩子数、叶层深度。只画出平衡外形会漏掉关键字重复或孩子接错；临时溢出框不是最终合法状态。

当前条件	操作与后续
w 红	w 染黑、p 染红，对 p 左旋；重新取 x 的兄弟，再处理下列黑兄弟情况
w 黑、两个孩子都黑	w 染红；若 p 红则 p 染黑并结束，否则将缺黑位置上移到 p 继续，移到根即可结束
w 黑、远侧黑、近侧红	近侧染黑、w 染红，对 w 右旋；重新取兄弟，转成下一行
w 黑、远侧红	w 继承 p 原颜色，p 与远侧染黑，对 p 左旋，结束缺黑修复

缺黑在右侧时，左右及旋转方向对称互换。最后一行的最小例子可写成 `10 黑 (缺黑 NIL,20 黑 (NIL,30 红))`：左旋 10 并换色，得到 `20 黑 (10 黑,30 黑)`，两侧向下的黑结点数相同。旋转前后中序仍为 10、20、30；修复既要补黑高，也要保持搜索次序。

这里颜色不表示访问频率，也不影响 BST 比较方向。做性质判断时，先检查中序有序，再检查根与 NIL、红红相邻、各分支黑高；仅数红黑结点总数不能判定一棵红黑树是否合法。

7.8 B 树：一个结点保存多个分隔关键字

本书 m 阶 B 树规定每结点最多 m 个孩子、最多 m−1 个关键字。除根外，每个数据结点至少有 t−1 个关键字，其中 $t = \lceil m/2 \rceil$；有非空孩子的非底层数据结点至少 t 个孩子，非叶根至少 2 个孩子、1 个关键字。最底层数据结点没有非空孩子，并且全部在同一层。若另外画查找失败的空外部叶，它们在下一层且不存关键字；以下高度不计这层空叶。

以下容量与高度公式采用 $m \geq 3$，高度为数据结点层数；“叶” 若按教材专指最下层的空失败结点，就不把它混入该高度。阶数 m、最小孩子数 $t = \lceil m/2 \rceil$ 和关键字数必须区分，不能把别处 “最小度数 t” 的算法原样按 m 替换。

一个结点有 k 个升序关键字，就有 k+1 个孩子区间。先在结点内找到目标或确定区间，再读相应孩子。多路分支降低外存层数；结点内可以顺序或折半比较，读盘次数不直接等于关键字比较次数。

7.8.1 插入从叶开始，分裂向上传播

以下是 **5 阶 B 树的教学机制例**，用紧凑关键字显示分裂与合并，不对应后面原题的阶数。5 阶结点最多 4 项，非根至少 2 项；非底层非根因此有 3 至 5 个孩子，非叶根仍可只有 2 个孩子。

初态根为 `[5,10,15,20]`，从左到右的 5 个叶分别为 `[1,3]`、`[6,8]`、`[11,13]`、`[16,18]`、`[21,23,25,27]`，全树共 16 个关键字。沿

搜索路径插入 29 后，最右叶临时有 5 项。提升第 3 项 25，叶分成 [21,23] 与 [27,29]，提升项从叶中移走。根因而临时成为 [5,10,15,20,25]，也有 5 项，不能到此结束。

再将根的第 3 项 15 提升为新根，左右内部结点分别为 [5,10]、[20,25]。原来的 6 个孩子按区间各接 3 个：左侧接 [1,3]、[6,8]、[11,13]，右侧接 [16,18]、[21,23]、[27,29]。两次分裂均为“5 项 → 左 2 项、提升 1 项、右 2 项”，内部结点分裂时还须同步分配孩子。根分裂使数据层数由 2 增至 3；局部叶分裂本身不增加全树高度。

最小反例·5 阶不能提前随意合并

5 阶非根至少 2 键、最多 4 键。两个合法最小兄弟各 2 键，加父分隔项共 5 键，已经溢出；删除后某侧只剩 1 键，才能按 1+1+2=4 合并。阶数约定决定合并时机。

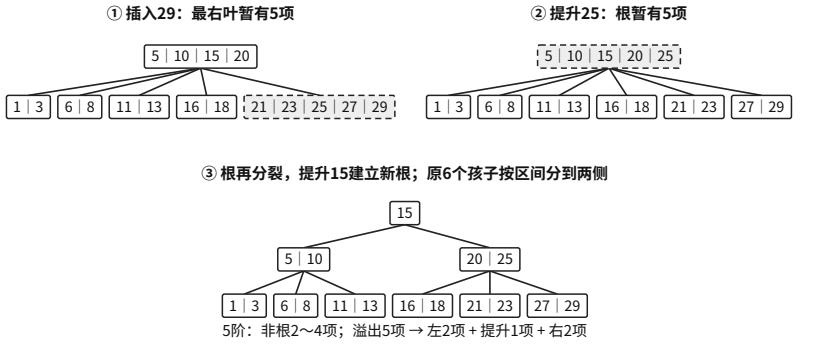


Figure 7.5: 5 阶 B 树插入 29，先分裂叶，再处理根溢出；虚框是待修复的 5 项结点

7.8.2 删除后的借位、合并与向上传播

删除内部关键字时，先用相邻前驱或后继关键字替换，再到最底层删除对应项。删除后若该结点的关键字少于最小数，先看相邻兄弟能否借：父分隔关键字下移，兄弟中最靠近当前结点的一项上移到父；若在内部层借位，右兄弟的最左孩子或左兄弟的最右孩子也要转接，保持孩子数等于关键字数加 1。兄弟也只有最小数时，合并当前结点、父分隔关键字与相邻兄弟，父可能继续下溢；空根仅剩一个孩子时，由孩子接替，高度减 1。

先看 5 阶借位小例：根 [10]，两个叶 [2,5] 与 [12,15,18]。删除 2 后左叶只剩 [5]，低于 2 项下限；右兄弟有 3 项，能够出借。将父 10 下移到左叶、右兄弟最小项 12 上移到父，得到根 [12]、叶 [5,10] 与 [15,18]。不能把 12 直接塞给左叶而让父仍为 10，否则分隔关系就被破坏。

再从插入图的最终树删除 1：左叶剩 [3]，相邻叶 [6,8] 已只有 2 项，不能借；合并它们与父分隔项 5，成为 [3,5,6,8]。其双亲原为 [5,10]，此时只剩 [10] 和 2 个孩子，也低于非根下限。右内部兄弟 [20,25] 同样不能借，故将左侧 [10]、根 15、右侧 [20,25] 合为 [10,15,20,25]，按顺序接 5 个原孩子。旧根为空，删除旧根后数据层数从 3 降为 2。

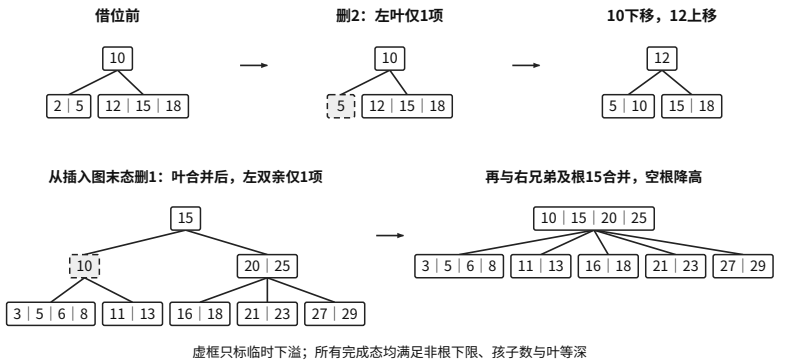


Figure 7.6: 上排为经父结点借位；下排为叶合并使双亲下溢，再合并并降低树高

5 阶 B 树的合并要特别注意时机：**两个各有 2 项的合法兄弟加父分隔项会有 5 项，超过上限**；本例是在删除造成一侧仅 1 项后，按 $1+1+2=4$ 项合并。不能把其他阶数或最小度参数下的提前合并规则直接套来。每个完成态都要复查结点内有序、孩子区间、关键字上下限、孩子数和叶等深；虚框中的临时下溢或溢出不算一个已完成的合法 B 树状态。

若高度 h 按数据结点层数计，非空 m 阶 B 树关键字总数满足 $2t^{h-1}-1 \le n \le m^h-1$，其中 $t=\lceil m/2 \rceil$。左端让根取 1 项、其余结点取最少分支，右端让各结点取满；求最大高度用左端，求最小高度用右端。

7.8.3 例 7-5　关键字总数怎样限制高度

例 7.8.1 (配套案例)

〖7.4-009〗一棵 3 阶 B 树共有 2047 个关键字，最大高度为（　），最小高度为（　）。

A. 11　B. 10　C. 8　D. 7

解析。高度按数据结点层数计。要使树尽量高，各层尽量稀疏：根 1 项、2 个孩子；其他结点至少 1 项、至少 2 个孩子。高度 h 时最少关键字 $1+2+\cdots+2^{h-1}=2^h-1$。因此 $2^h-1 \le 2047=2^{11}-1$，最大 11，第一空 **A**。

要使树尽量矮，各结点取满，每层关键字数分别 2、6、18……，高度 h 时最多 3^h-1。$3^6-1=728<2047\le 3^7-1=2186$，所以最小 7，第二空 **D**。这两端分别使用 “最少容量” 和 “最大容量”，不能将不等式方向颠倒。

7.8.4 例 7-6　小规模 B 树的形态怎样完整计数

例 7.8.2 (配套案例)

【配套例题】给定 7 个不同关键字，能构造的不同 4 阶 B 树的个数最多为（　）。

A. 7　B. 8　C. 9　D. 10

解析。每结点最多 3 项，非根至少 1 项；关键字的升序分配由形态与各结点项数确定，不再任意排列。

高度 1 最多 3 项，不够。高度 2 时，设根有 r 项，则有 r+1 个叶，叶

不变量·B 树合法性有五项

每次操作完成后同时检查结点内有序、孩子区间、键数上下限、孩子数、叶层深度。只画出平衡外形会漏掉关键字重复或孩子接错；临时溢出框不是最终合法状态。

项数和为 7−r，各叶 1 至 3 项：r=1 时只有 `(3,3)`，1 种；r=2 时是正整数三元组和 5，各项至多 3，包括 `(3,1,1)` 的 3 个排列与 `(2,2,1)` 的 3 个排列，共 6 种；r=3 时只有 `(1,1,1,1)`，1 种。因此高度 2 共 8 种。

高度 3 时，最少项数为 $1+2+4=7$，只能每结点 1 项且每个内部结点 2 个孩子，1 种。高度 4 最少 15 项，已不可能。合计 $8+1=9$，答案为 **C**。只考虑根的取值而不枚举叶的分配，会漏掉不同的结点分组。

课程连接·索引层服务外存页访问

B+ 内部省去整条记录，通常能在同页放更多索引项，降低层数；叶链便于范围扫描。外存优势取决于页大小、缓存和记录存放，不能改写成任何输入均比内存 BST 快。

7.9 B+ 树：索引层与叶层分工

B+ 树把记录或记录指针集中在叶结点，内部结点保存索引关键字及孩子指针；叶结点按关键字有序，并用链接便于连续访问。一次点查找沿索引降到叶；范围查找先定位起点叶，再沿叶链扫描，避免对每个值重新从根下降。

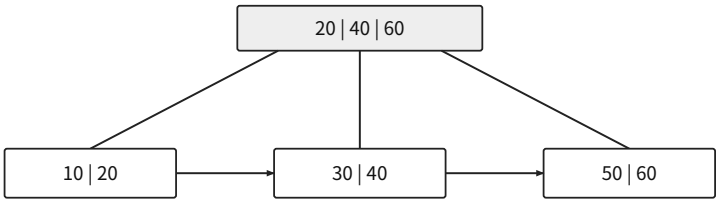


Figure 7.7: 内部层只导航，叶层保存全部关键字并按序相连

本章按王道的 “内部结点每个关键字对应一棵子树最大关键字” 约定。非叶根至少有 2 棵子树，其余分支结点至少有 $\lceil m/2 \rceil$ 棵子树，最多均为 m 棵；分支结点的关键字数与子树数相同。本图中，索引 `[20|40|60]` 指向最大值分别为 20、40、60 的三片子树。做题应先按给定图或定义确定，不把 B 树的 “k 键必有 k+1 孩子” 无条件套给 B+ 树。

B 树的查找可以在内部结点命中时结束；B+ 树的记录查询通常仍要走到叶。叶链让 B+ 适合区间访问，并不意味着每次点查都顺着整条叶链查找。内部不携带整条记录，通常能提高同样页大小下的分支数，减少外存层数；优势来自页组织与访问方式，不能据此声称任何规模都一定比内存 BST 更快。

7.10 散列表：冲突、删除与 ASL

散列函数 $H(key)$ 把关键字映射到表地址；不同关键字有相同函数值称为同义词，发生地址冲突。函数应对同一关键字给出确定地址，并尽量均匀分散。随机改变同一关键字的地址会使后续查找无法复现位置，不能直接作为稳定散列函数。

7.10.1 散列函数的构造条件

直接定址可用 $H(key)=a\,key+b$，适合关键字分布较连续且范围可控；它避免了所选模型内的碰撞，但地址范围大时空间可能浪费。除留余数法用 $H(key)=key \bmod p$，通常选不大于表长且有利于分布的素数 p；取模常数与开放定址继续探查的表长 m 是两个参数。

数字分析法利用已有关键字分布选择较均匀的数位；平方取中法先平方再取中间若干位，希望让多位信息影响结果。没有一种构造能对所有未知输入自动保证无冲突或均匀，应依据关键字类型、分布和地址范围选择；同一关键字必须能稳定复现相同初始地址。

开放定址在表内继续探查空位置。线性探查地址为 $(H_0+k) \bmod m$；二次探查可能规定为 $(H_0+k^2) \bmod m$，也可能按正负平方交替，须按题目公式执行。探查时的模数取表长 m；散列函数内部的除数 p 可能小于 m，两者不能混用。线性探查能够在一整轮内访问全表，二次探查不一定遍历所有位置，即使表未满也可能找不到可达空位。

链地址法把同义词放在一个桶的链表中，成功时沿链比较，失败时须比较到链尾。开放定址的聚集不仅来自同义词，也可能由不同初始地址的探查链交错引起。装填因子 $\alpha = n/m$ 越大，一般可用位置越少；散列并不保证任何输入下都严格常数时间。

条件辨析·互质条件检查实际步长

素数表长也不能挽救模表长为 0 的步长。步长须在 1 至 m-1 范围且与 m 互质，才能覆盖全表。

7.10.2 探测公式与覆盖范围

开放定址统一写作 $H_i = (H(key)+d_i) \bmod m$。增量每次加在**初始地址**上，各方法的规则如下：

方法	增量或步长	完整探测的条件
线性探测	$d_i = i$	连续 m 次能遍历全表
平方探测	0、1、−1、4、−4、9、−9、⋯	依表长与增量范围；有空位未必能到达
双散列	$d_i = i \cdot hash_2(key)$	步长与 m 互质时遍历全表
伪随机探测	题目规定的可复现增量序列	需实际检查增量模 m 覆盖的剩余类

当 m 为形如 $4j+3$ 的素数并采用足够的正负平方增量时，可覆盖全部位置。若只用正平方序列，则是另一套覆盖条件，不能把这条结论直接移用。

双散列需保证步长模 m 非零且与 m 互质。例如 $m=8$、步长 2 只能访问 4 个位置，步长 3 能访问全部 8 个位置。即使 m 为素数，若取 `hash2(key)=m-(key%m)`，在 `key%m==0` 时步长等于 m，仍会反复停在原地址；可在整数关键字场景采用 `1+(key%(m-1))`产生 $1..m-1$ 的步长，再明确负数取模规则。

7.10.3 链地址法的完整例

课件取表长 13、$H(key) = key \bmod 13$，依次插入 `19,14,23,1,68,20,84,27,55,11,10,79`。若采用头插，非空桶为：

桶号	从首结点到尾结点的关键字
1	79→27→1→14
3	55→68
6	84→19
7	20
10	10→23
11	11

查 27 比较 79、27，共 2 次；查 66 落桶 1，比较 4 个关键字后失败；查 21 落空桶 8，关键字比较次数为 0。若题目统计空指针判断，则另计。等概率成功 ASL 为 $(10+3+3+1+3+1)/12 = 21/12$；13 个桶等概

率失败时 ASL 为 12/13，因为桶长之和为 12。采用尾插会改变某一具体记录的成功路径，等概率成功 ASL 对每桶长度固定的全体记录平均仍相同。

课件还讨论了桶内保持关键字有序的链表：递增链中，查找值小于当前关键字时即可提前失败，有机会减少失败比较；代价是插入时也要维护有序位置。具体 ASL 须按新链序和目标值分布重新计算，不能继续沿用 “所有失败都扫描到链尾” 的口径。

条件辨析·分母由失败事件决定

表长 16 且 H 取模 13 时，初始地址只有 13 种。探测经过的地址和初始地址集合不同，平均时不能见表长就除 16。

7.10.4 表长与散列值域不同的 ASL

同一关键字序列放入长度 16 的开放定址表，$H(key) = key \bmod 13$，冲突时以表长 16 作线性探测。最终 0 号空，1～12 号依次为 `14,1,68,27,55,19,20,84,79,23,11,10`，13～15 号空。成功探测次数依插入次序为 `1,1,1,2,1,1,3,4,3,1,3,9`，成功 ASL 为 $30/12 = 2.5$。

失败事件若按**初始地址 0～12 等概率**，对应探测长度为 `1,13,12,11,10,9,8,7,6,5,4,3,2`，故失败 ASL 为 $91/13 = 7$。13～15 号仍可在探测过程中经过，但散列函数不会把它们作为初始地址；分母应为 13 而不是 16。删除 20 并保留删除标记后，成功记录变 11 项，成功 ASL 为 29/11，失败路径仍不被截断，失败 ASL 保持 7。

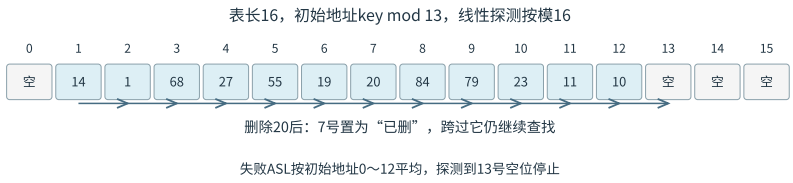


Figure 7.8: 同一探测链跨过删除标记；初始散列地址只来自 0 至 12

7.10.5 例 7-7　删除标记为什么继续计入失败路径

例 7.10.1 (配套案例)

【配套例题】长度为 5、初始为空的散列表 HT，$H(k) = (k + 4) \bmod 5$，线性探查处理冲突。依次插入 2022、12、25，再删除 25，查找失败的平均查找长度为（　）。

A. 1　B. 1.6　C. 1.8　D. 2.2

解析。 2022 与 12 初始地址都为 1，分别存 1 与 2；25 存 4。删除 4 处关键字后应留 “已删除” 标记，而不是把它当作从未使用的空位置。得到：

地址	槽状态	从此地址起失败探查次数
0	从未用空	1
1	2022	3
2	12	2
3	从未用空	1
4	已删除	2

失败查找遇到从未使用的空位才结束；越过删除标记还要继续。按 5 个初始地址等概率，ASL 为 $(1 + 3 + 2 + 1 + 2)/5 = 1.8$，答案为 **C**。探查到空单元也计一次表访问，这是本题失败 ASL 的计数口径；删除标记处没有有效关键字可比，却不能在这里终止。

若把一个探查链的中间位置直接清空，后面的元素虽然还在，查找却会在这里提前失败。插入时删除位置可以复用；若不允许重复关键字，要继续查到能排除已有同关键字的位置，再决定使用哪个已删除槽。

不变量·墓碑保住后续探查路径

已删除槽说明这里曾发生占用，后面可能仍有沿同一序列插入的元素。查找只能在从未用空槽或探查耗尽处失败；插入复用墓碑前还要排除后方已有同关键字。

7.10.6 例 7-8　二次探查必须回到原始地址计算

例 7.10.2 (配套案例)

【配套例题】将 20、3、11、18、9、14、7 依次插入初始为空、长度 11 的 HT。$H(key) = (key \times 3) \bmod 11$，初始地址为 H_0，冲突时 $H_k = (H_0 + k^2) \bmod 11$ $(k = 1, 2, \ldots)$。

（1）画出 HT 并计算装填因子。（2）给出查找 14 时的关键字比较序列。（3）查找 8，确认失败时的散列地址是多少?

解析。 20、3、11、18 直接存于 5、9、0、10。9 从 5 冲突后到 6；14 从 9 依次探查 10、2，存 2；7 从 10 依次探查 0、3，存 3。

地址	关键字
0	11
1	空
2	14
3	7
4	空
5	20
6	9

地址	关键字
7	空
8	空
9	3
10	18

装填因子为 7/11。查 14 的探查地址为 9、10、2，比较关键字为 3、18、14。查 8 时，$H_0 = 24 \bmod 11 = 2$，探查地址依次为 2、3、6、0、7，第 7 号为空，所以在**地址 7** 确认失败。每个新地址从同一个 H_0 加平方增量计算，不能在上一次地址上再加 k^2。

7.10.7 开放定址的三种槽状态

```
hashFind(key):
    按约定的探查序列访问地址
    若槽为有效且关键字相同：成功
    若槽从未使用：失败
    若槽已删除或关键字不同：继续探查
    若可探查位置已耗尽：失败
```

删除先按同一路径找到关键字，再把槽改为 “已删除”。插入可记住第一个已删除槽，但不能立即停在那里：若后面已有同关键字且不允许重复，提前写入会制造重复。探查总步数须有界，表满或二次探查可达集合已走完时不能无限循环。

链地址法的插入在相应桶链完成，删除是真实断链并释放目标结点；它不需要开放定址的墓碑标记。链长才是比较量来源，不能拿开放定址中 “遇从未用空槽终止” 的规则解释链地址失败。

不变量·墓碑保住后续探查路径

已删除槽说明这里曾发生占用，后面可能仍有沿同一序列插入的元素。查找只能在从未用空槽或探查耗尽处失败；插入复用墓碑前还要排除后方已有同关键字。

7.11 怎样按操作约束选择方法

下表把主要成立条件与查找代价分开。记录数为 n，散列表长为 m，树高记为 h；B/B+ 树的 h 按数据层数计，假定每个结点占一个外存页，表中的读盘上界指索引结点路径上的页访问，还未扣除已缓存页；若叶中只存记录指针，随后取叶外记录的 I/O 另计。

方法或结构	成立条件	查找代价	更新、范围或外存特点
顺序查找	可无序；数组或链表	最坏 $O(n)$	不需维护有序；范围筛选仍须扫描
折半查找	有序且能随机定位的顺序表	最坏 $O(\log n)$	数组插删最坏移动 $O(n)$；可定位区间端点
普通 BST	满足全部子树的区间关系	$O(h)$，最坏 $O(n)$	支持前驱后继；无平衡维护时可退化成链
AVL	BST 且任一结点两侧高差至多 1	最坏 $O(\log n)$	动态插删需维护高度与旋转
红黑树	BST 及红黑性质均成立	最坏 $O(\log n)$	动态插删维护颜色、黑高与旋转
B 树	多关键字有序、容量约束、叶等深	至多 h 次结点读盘	内部命中可停；读入结点后的键比较另计
B+ 树	按给定索引约定，叶层存全体关键字及记录或记录指针	点查沿根到叶，至多 h 次索引页读盘	定位起点后沿叶链作范围访问

方法或结构	成立条件	查找代价	更新、范围或外存特点
散列表	散列函数、装填因子和冲突策略匹配	合适散列与受控负载下平均 $O(1)$；最坏链扫描 $O(n)$，开放定址探查 $O(m)$	扩容有维护成本；原生不保序，不直接支持有序范围定位

一个表可能有多种索引。不能只为某一次查找更快，就忽略频繁更新带来的维护成本；也不能用散列的平均等值查找优势替代区间检索所需的有序性。题目给出 “有序、随机访问、频繁插删、外存、范围” 时，分别把它们转成结构成立条件与访问成本，再作选择。

7.12 主动回忆自测

以下为自拟补充，检查维护过程与失败口径。

1. BST 中每个结点都满足 “左孩子小于自身、右孩子大于自身”，是否足够？给反例。
2. 4 阶 B 树一个非根结点最多、最少有多少关键字？根为空时能否直接认定整树空？
3. 一条散列探查路径经过 “有效关键字—删除标记—有效关键字—从未用空位”，目标均不等，失败探查几次？
4. 红黑树根 10 黑、孩子 5 红和 15 红，插入 1 后，首先应旋转还是变色？

5. AVL 删除后只修复最深一处失衡是否总够？与插入有何不同？
6. B 树一个内部结点有 3 个关键字，按本章约定有几个孩子区间？B+ 树能无条件沿用吗？
7. 在有序链表上执行折半式比较，为何不能直接承诺总时间 $O(\log n)$？
8. 为什么散列表失败 ASL 的分母未必是有效记录数？若表长 5、只存 2 项，按初始地址等概率应除以几？

答案。1. 不够；根 10，左孩子 5，5 的右孩子 12，所有直接比较都满足，但 12 在 10 左子树却大于 10。2. 最多 3 项、最少 1 项；根空但还有唯一孩子时应降层，由孩子接替，树不空。3. 4 次，删除标记不能截断路径。4. 先变色，叔 15 红，把 5、15 变黑、10 暂红，最后根恢复黑，无需该处旋转。

5. 不总够；删除可能让修复后子树高度继续减 1，导致更高祖先失衡。插入修复最小失衡子树后通常恢复原高度，可阻断继续传播。
6. 有 4 个孩子区间；B+ 索引约定各异，若每项表示一棵子树最大关键字，键数和孩子数可以相等，需按给定定义。
7. 比较次数可能为对数，但链表不能常数时间随机定位中点，节点访问成本没有消失。
8. 失败事件分布可以是初始散列地址或失败区间，而非已有记录；该例应按 5 个初始地址求平均，除以 5。

不变量·墓碑保住后续探查路径

已删除槽说明这里曾发生占用，后面可能仍有沿同一序列插入的元素。查找只能在从未用空槽或探查耗尽处失败；插入复用墓碑前还要排除后方已有同关键字。